Lab 10 — Fairness and Robustness Audit

Track B (BMD-45 vehicle detection) · Week 10 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/fixtures/` with your own synthetic CCTV images — same two camera profiles and per-class size multipliers as everyone else's, seeded deterministically from your student ID. Because this lab's whole point is a recall disparity between small-vehicle classes and everything else, and that disparity can occasionally wash out at 60-images-per-student scale, the script checks its own candidate output with the actual detector and retries with a new sub-seed until Bicycle and Two-wheeler both come out below 90% recall while no large class does — so the disparity survives reseeding no matter which student ID you use.

Other mid-sized classes (for example LCV or MUV) may ALSO land below 90% for some student IDs. Each class has only about 5-30 ground-truth boxes, so a recall of 86% vs 92% can be sampling noise, not a real gap. That is expected, not a bug, and your `NOTES.md` should say whether you think each flagged class is a real disparity or noise.

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed.

Files

- `src/fairness_audit.py` — implement the four `# TODO` functions.
- `src/adversarial.py` — implement the three `# TODO` functions (attack half).
- `src/mock_detector.py` — dependency-light stand-in detector (provided, don't edit).
- `src/run_pipeline.py` — complete driver script. Don't edit.
- `prep_data.py` — superseded, don't run (see `generate_for_student.py`).
- `data/fixtures/` — your personalized synthetic images across BMD-45's 14 vehicle classes, generated above (don't hand-edit).

A note on framing

Real fairness audits (Fairlearn, demographic parity, equalized odds) measure disparity across PROTECTED DEMOGRAPHIC GROUPS. BMD-45 is a vehicle-detection dataset with no demographic attributes, so this lab applies the same underlying methodology — measure per group, flag underperformers — to vehicle CLASSES instead. The code and statistics are identical to a real fairness audit; only the grouping variable differs. `src/fairness_audit.py`'s docstring says this too — it's worth actually reading, not just skimming, since your `NOTES.md` will ask about it.

Background

`data/fixtures/` (from `generate_for_student.py` above — `prep_data.py` is superseded, don't run it) has 60 synthetic images across BMD-45's 14 vehicle classes, built so smaller-vehicle classes (bicycles, two-wheelers) get systematically smaller drawn boxes — the same real pattern BMD-45's paper documents: small object classes are structurally harder for a detector to pick up. The metric is per-class RECALL: of all ground-truth vehicles of a given class, what fraction did the detector find at all (matched by bounding-box overlap, not by the detector's own — unreliable — class guess)?

Your task

`src/fairness_audit.py` (four functions marked `# TODO`, each has a full docstring spec, ~40 min)

- `compute_iou(box_a, box_b)` — Intersection-over-Union between two boxes.
- `match_ground_truth(gt_boxes, detections, iou_threshold)` — was each ground-truth box found by any detection?
- `audit_by_category(coco_path, fixtures_dir, iou_threshold)` — run the detector over every image, accumulate per-class recall.
- `find_underperforming_categories(category_stats, recall_threshold)` — surface the classes below the bar, worst first.

`src/adversarial.py` (three functions marked `# TODO`, each has a full docstring spec, ~20 min)

- `perturb_image(img, eps)` — FGSM-style white-box perturbation: lower red, raise green and blue by at most `eps` per pixel. The detector's colour rule is readable, so the "gradient sign" is obvious.
- `audit_under_attack(coco_path, fixtures_dir, eps_values)` — re-run the audit at each budget (use the `transform` argument of `audit_by_category`).
- `find_most_fragile(attack_stats, eps, top_k)` — rank classes by recall drop.

The mock detector is a hand-written colour rule, not a CNN, so it is far more brittle than a real model. The method transfers; the exact numbers do not.

```
python src/run_pipeline.py
```

Writes `fairness_report.json` with per-class recall and the flagged underperforming classes (`src/run_pipeline.py` is complete, don't edit it).

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader.

Deliverables (what you commit)

- `src/fairness_audit.py` and `src/adversarial.py`, completed.
- `fairness_report.json` (from running the pipeline).
- A short `NOTES.md`: the `--student-id` value you used (required — see above), which classes got flagged, does that match what you'd expect given how `generate_for_student.py` built the fixtures, and — tying back to the framing note above — what would change about this audit's stakes and your obligations as an engineer if the grouping variable were a protected demographic attribute instead of a vehicle class? One more: pick one missed ground-truth box in a flagged class. An auditor asks you to explain WHY the detector missed it, not just that it did. Would SHAP or LIME actually help you answer that for a bounding-box detector — and if not, what's different about this detector's inputs/outputs that breaks the assumption both tools make? Last: name one lifecycle stage and attack from the lecture (data poisoning, model theft, or adversarial examples) that could silently corrupt THIS audit's result, and one control from the lecture that would reduce the risk. Finally, using your attack results: at what `eps` does each flagged class fall further, which class is most fragile, is it the same class the clean audit flagged, how does the biggest-drop ranking differ from the lowest-recall-under-attack ranking, and what would adversarial training or input validation look like for this detector?

Grading checklist

- [ ] `data/fixtures/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- [ ] `compute_iou` is correct on identical, non-overlapping, and partially-overlapping boxes (checked with held-out cases beyond the ones in `tests/test_smoke.py`).
- [ ] `match_ground_truth` correctly marks covered vs. uncovered boxes.
- [ ] `audit_by_category` correctly skips zero-count categories and computes recall correctly.
- [ ] `find_underperforming_categories` sorts worst-first and respects the threshold as a strict inequality.
- [ ] `perturb_image` shifts channels by exactly `eps` in the right directions, clamps to [0, 255], and does not mutate its input.
- [ ] `audit_under_attack` returns clean stats at eps=0 and uses `audit_by_category`'s `transform`; `find_most_fragile` ranks by recall drop with alphabetical tie-break.
- [ ] `NOTES.md` engages seriously with the framing question and the SHAP/LIME question, not just a one-line dismissal, and names a concrete attack and control for the lifecycle question.
- [ ] Meaningful commit history and a working README.

Submission

```
git add -A
git commit -m "Week 10: fairness and robustness audit"
git tag week10-submit
git push origin main --tags
```

Source: this lab operationalizes the Bias Detection and Fairness Monitoring content from the Week 10 lecture deck. Dataset: Sharma et al., "BMD-45: A Large-Scale CCTV Vehicle Detection Dataset for Urban Traffic," CVPR 2026 Findings.